1
Thread Management

In this chapter, we will cover the following topics:

- Creating, running, and setting the characteristics of a thread
- Interrupting a thread
- Controlling the interruption of a thread
- Sleeping and resuming a thread
- Waiting for the finalization of a thread
- Creating and running a daemon thread
- Processing uncontrolled exceptions in a thread
- Using thread local variables
- Grouping threads and processing uncontrolled exceptions in a group of threads
- Creating threads through a factory

Introduction

In the computer world, when we talk about concurrency, we refer to a series of independent and unrelated tasks that run simultaneously on a computer. This simultaneity can be real if the computer has more than one processor or a multi-core processor, or it can be apparent if the computer has only one core processor.

All modern operating systems allow the execution of concurrent tasks. You can read your e-mails while listening to music or reading news on a web page. We can say this is process-level concurrency. But inside a process, we can also have various simultaneous tasks. Concurrent tasks that run inside a process are called **threads**. Another concept related to concurrency is parallelism. There are different definitions and relations with the concurrency concept. Some authors talk about concurrency when you execute your application with multiple threads in a single-core processor. With this, you can see when your program execution is apparent. They talk about parallelism when you execute your application with multiple threads in a multi-core processor or in a computer with more than one processor, so this case is real as well. Other authors talk about concurrency when the threads of an application are executed without a predefined order, and they discuss parallelism when all these threads are executed in an ordered way.

This chapter presents a number of recipes that will show you how to perform basic operations with threads, using the Java 9 API. You will see how to create and run threads in a Java program, how to control their execution, process exceptions thrown by them, and how to group some threads to manipulate them as a unit.

Creating, running, and setting the characteristics of a thread

In this recipe, we will learn how to do basic operations over a thread using the Java API. As with every element in the Java language, threads are objects. We have two ways of creating a thread in Java:

- Extending the `Thread` class and overriding the `run()` method.
- Building a class that implements the `Runnable` interface and the `run()` method and then creating an object of the `Thread` class by passing the `Runnable` object as a parameter--this is the preferred approach and it gives you more flexibility.

In this recipe, we will use the second approach to create threads. Then, we will learn how to change some attributes of the threads. The `Thread` class saves some information attributes that can help us identify a thread, know its status, or control its priority. These attributes are:

- **ID**: This attribute stores a unique identifier for each thread.
- **Name**: This attribute stores the name of the thread.
- **Priority**: This attribute stores the priority of the `Thread` objects. In Java 9, threads can have priority between 1 and 10, where 1 is the lowest priority and 10 is the highest. It's not recommended that you change the priority of the threads. It's only a hint to the underlying operating system and it doesn't guarantee anything, but it's a possibility that you can use if you want.
- **Status**: This attribute stores the status of a thread. In Java, a thread can be present in one of the six states defined in the `Thread.State` enumeration: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, or `TERMINATED`. The following is a list specifying what each of these states means:
    - `NEW`: The thread has been created and it has not yet started
    - `RUNNABLE`: The thread is being executed in the JVM
    - `BLOCKED`: The thread is blocked and it is waiting for a monitor
    - `WAITING`: The thread is waiting for another thread
    - `TIMED_WAITING`: The thread is waiting for another thread with a specified waiting time
    - `TERMINATED`: The thread has finished its execution

In this recipe, we will implement an example that will create and run 10 threads that would calculate the prime numbers within the first 20,000 numbers.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Calculator` that implements the `Runnable` interface:

```
public class Calculator implements Runnable {
```

2. Implement the `run()` method. This method will execute the instructions of the thread we are creating, so this method will calculate the prime numbers within the first `20000` numbers:

```
@Override
public void run() {
  long current = 1L;
  long max = 20000L;
  long numPrimes = 0L;

  System.out.printf("Thread '%s': START\n",
                    Thread.currentThread().getName());
  while (current <= max) {
    if (isPrime(current)) {
      numPrimes++;
    }
    current++;
  }
  System.out.printf("Thread '%s': END. Number of Primes: %d\n",
                  Thread.currentThread().getName(), numPrimes);
}
```

3. Then, implement the *auxiliar* `isPrime()` method. This method determines whether a number is a prime number or not:

```
private boolean isPrime(long number) {
  if (number <= 2) {
    return true;
  }
  for (long i = 2; i < number; i++) {
    if ((number % i) == 0) {
      return false;
    }
  }
  return true;
}
```

4. Now implement the main class of the application. Create a class named `Main` that contains the `main()` method:

```
public class Main {
        public static void main(String[] args) {
```

5. First, write some information regarding the values of the maximum, minimum, and default priority of the threads:

```
System.out.printf("Minimum Priority: %s\n",
                  Thread.MIN_PRIORITY);
System.out.printf("Normal Priority: %s\n",
                  Thread.NORM_PRIORITY);
System.out.printf("Maximun Priority: %s\n",
                  Thread.MAX_PRIORITY);
```

6. Then create 10 `Thread` objects to execute 10 `Calculator` tasks. Also, create two arrays to store the `Thread` objects and their statuses. We will use this information later to check the finalization of the threads. Execute five threads (the even ones) with maximum priority and the other five with minimum priority:

```
Thread threads[];
Thread.State status[];
threads = new Thread[10];
status = new Thread.State[10];
for (int i = 0; i < 10; i++) {
threads[i] = new Thread(new Calculator());
  if ((i % 2) == 0) {
    threads[i].setPriority(Thread.MAX_PRIORITY);
  } else {
    threads[i].setPriority(Thread.MIN_PRIORITY);
  }
    threads[i].setName("My Thread " + i);
}
```

7. We are going to write information in a text file, so create a try-with-resources statement to manage the file. Inside this block of code, write the status of the threads in the file before you launch them. Then, launch the threads:

```
try (FileWriter file = new FileWriter(".\\data\\log.txt");
PrintWriter pw = new PrintWriter(file);) {

  for (int i = 0; i < 10; i++) {
    pw.println("Main : Status of Thread " + i + " : " +
                threads[i].getState());
    status[i] = threads[i].getState();
```

```
    }
    for (int i = 0; i < 10; i++) {
      threads[i].start();
    }
```

8. After this, wait for the finalization of the threads. As we will learn in the *Waiting for the finalization of a thread* recipe of this chapter, we can use the `join()` method to wait for this to happen. In this case, we want to write information about the threads when their statuses change, so we can't use this method. We use this block of code:

```
      boolean finish = false;
      while (!finish) {
        for (int i = 0; i < 10; i++) {
          if (threads[i].getState() != status[i]) {
            writeThreadInfo(pw, threads[i], status[i]);
            status[i] = threads[i].getState();
          }
        }

        finish = true;
        for (int i = 0; i < 10; i++) {
          finish = finish && (threads[i].getState() ==
                              State.TERMINATED);
        }
      }

    } catch (IOException e) {
      e.printStackTrace();
    }
  }
```

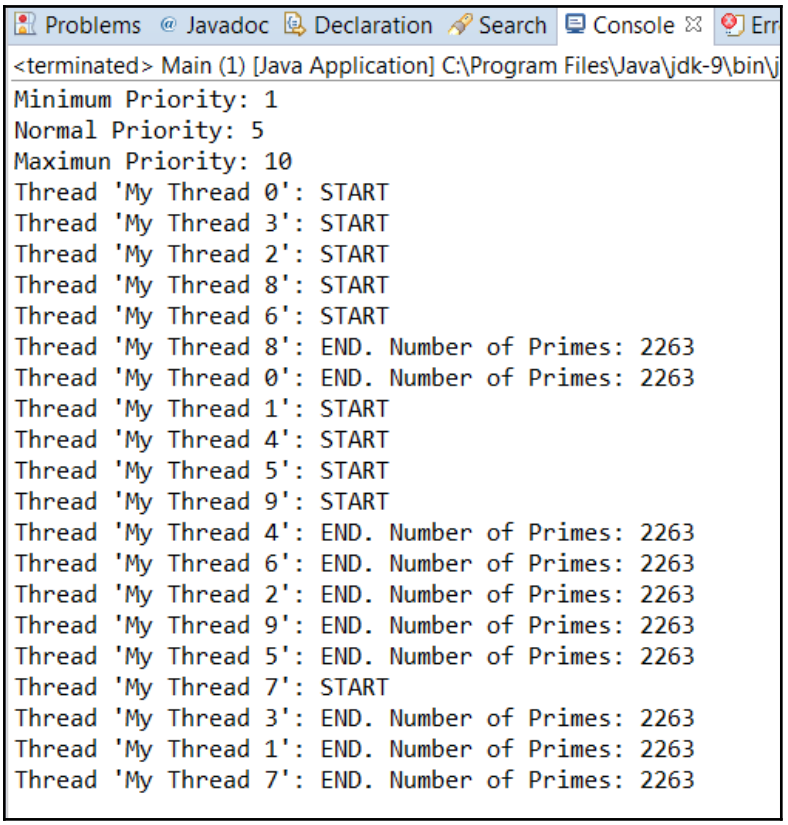
9. In the previous block of code, we called the `writeThreadInfo()` method to write information about the status of a thread in the file. This is the code for this method:

```
  private static void writeThreadInfo(PrintWriter pw,
                                      Thread thread,
                                      State state) {
    pw.printf("Main : Id %d - %s\n", thread.getId(),
              thread.getName());
    pw.printf("Main : Priority: %d\n", thread.getPriority());
    pw.printf("Main : Old State: %s\n", state);
    pw.printf("Main : New State: %s\n", thread.getState());
    pw.printf("Main : ************************************\n");
  }
```

10. Run the program and see how the different threads work in parallel.

How it works...

The following screenshot shows the console part of the output of the program. We can see that all the threads we have created run in parallel to do their respective jobs:



In this screenshot, you can see how threads are created and how the ones with an even number are executed first, as they have the highest priority, and the others executed later, as they have minimum priority. The following screenshot shows part of the output of the `log.txt` file where we write information about the status of the threads:

```
 6Main : Status of Thread 5 : NEW
 7Main : Status of Thread 6 : NEW
 8Main : Status of Thread 7 : NEW
 9Main : Status of Thread 8 : NEW
10Main : Status of Thread 9 : NEW
11Main : Id 13 - My Thread 0
12Main : Priority: 10
13Main : Old State: NEW
14Main : New State: RUNNABLE
15Main : **********************************
16Main : Id 14 - My Thread 1
17Main : Priority: 1
18Main : Old State: NEW
19Main : New State: BLOCKED
20Main : **********************************
21Main : Id 15 - My Thread 2
22Main : Priority: 10
23Main : Old State: NEW
24Main : New State: RUNNABLE
25Main : **********************************
```

Every Java program has at least one execution thread. When you run the program, JVM runs the execution thread that calls the `main()` method of the program.

When we call the `start()` method of a `Thread` object, we are creating another execution thread. Our program will have as many execution threads as the number of calls made to the `start()` method.

The `Thread` class has attributes to store all of the information of a thread. The OS scheduler uses the priority of threads to select the one that uses the CPU at each moment and actualizes the status of every thread according to its situation.

If you don't specify a name for a thread, JVM automatically assigns it one in this format: Thread-XX, where XX is a number. You can't modify the ID or status of a thread. The `Thread` class doesn't implement the `setId()` and `setStatus()` methods as these methods introduce modifications in the code.

A Java program ends when all its threads finish (more specifically, when all its non-daemon threads finish). If the initial thread (the one that executes the `main()` method) ends, the rest of the threads will continue with their execution until they finish. If one of the threads uses the `System.exit()` instruction to end the execution of the program, all the threads will end their respective execution.

Creating an object of the `Thread` class doesn't create a new execution thread. Also, calling the `run()` method of a class that implements the `Runnable` interface doesn't create a new execution thread. Only when you call the `start()` method, a new execution thread is created.

There's more...

As mentioned in the introduction of this recipe, there is another way of creating a new execution thread. You can implement a class that extends the `Thread` class and overrides the `run()` method of this class. Then, you can create an object of this class and call the `start()` method to have a new execution thread.

You can use the static method `currentThread()` of the `Thread` class to access the thread object that is running the current object.

You have to take into account that the `setPriority()` method can throw an `IllegalArgumentException` exception if you try to establish priority that isn't between 1 and 10.

See also

- The *Creating threads through a factory* recipe of this chapter

Interrupting a thread

A Java program with more than one execution thread only finishes when the execution of all of its threads end (more specifically, when all its non-daemon threads end their execution or when one of the threads uses the `System.exit()` method). Sometimes, you may need to finish a thread because you want to terminate a program or when a user of the program wants to cancel the tasks that a thread object is doing.

Java provides an interruption mechanism that indicates to a thread that you want to finish it. One peculiarity of this mechanism is that thread objects have to check whether they have been interrupted or not, and they can decide whether they respond to the finalization request or not. A thread object can ignore it and continue with its execution.

In this recipe, we will develop a program that creates a thread and forces its finalization after 5 seconds, using the interruption mechanism.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class called `PrimeGenerator` that extends the `Thread` class:

```
public class PrimeGenerator extends Thread{
```

2. Override the `run()` method including a loop that will run indefinitely. In this loop, process consecutive numbers beginning from one. For each number, calculate whether it's a prime number; if yes, as in this case, write it to the console:

```
@Override
public void run() {
  long number=1L;
  while (true) {
    if (isPrime(number)) {
      System.out.printf("Number %d is Prime\n",number);
    }
```

3. After processing a number, check whether the thread has been interrupted by calling the `isInterrupted()` method. If this method returns `true`, the thread has been interrupted. In this case, we write a message in the console and end the execution of the thread:

```
      if (isInterrupted()) {
        System.out.printf("The Prime Generator has been
                          Interrupted");
        return;
      }
      number++;
    }
  }
```

4. Implement the `isPrime()` method. You can get its code from the *Creating, running, and setting information of a thread* recipe of this chapter.
5. Now implement the main class of the example by implementing a class called `Main` and the `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

6. Create and start an object of the `PrimeGenerator` class:

```
Thread task=new PrimeGenerator();
task.start();
```

7. Wait for 5 seconds and interrupt the `PrimeGenerator` thread:

```
try {
  Thread.sleep(5000);
} catch (InterruptedException e) {
  e.printStackTrace();
}
task.interrupt();
```

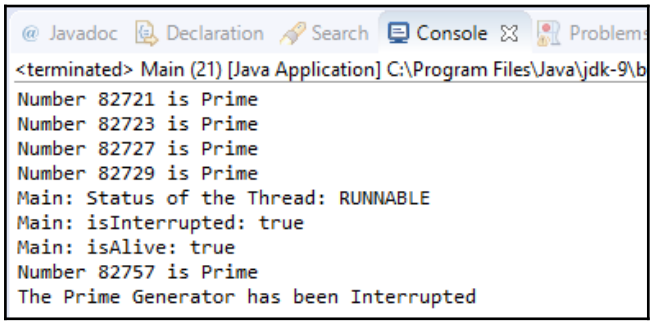
8. Then, write information related to the status of the interrupted thread. The output of this piece of code will depend on whether the thread ends its execution before or after:

```
        System.out.printf("Main: Status of the Thread: %s\n",
                          task.getState());
        System.out.printf("Main: isInterrupted: %s\n",
                          task.isInterrupted());
        System.out.printf("Main: isAlive: %s\n", task.isAlive());
    }
```

9. Run the example and see the results.

How it works...

The following screenshot shows the result of the execution of the previous example. We can see how the `PrimeGenerator` thread writes the message and ends its execution when it detects that it has been interrupted. Refer to the following screenshot:



The `Thread` class has an attribute that stores a `boolean` value indicating whether the thread has been interrupted or not. When you call the `interrupt()` method of a thread, you set that attribute to `true`. The `isInterrupted()` method only returns the value of that attribute.

The `main()` method writes information about the status of the interrupted thread. In this case, as this code is executed before the thread has finished its execution, the status is `RUNNABLE`, the return value of the `isInterrupted()` method is `true`, and the return value of the `isAlive()` method is `true` as well. If the interrupted `Thread` finishes its execution before the execution of this block of code (you can, for example, sleep the main thread for a second), the methods `isInterrupted()` and `isAlive()` will return a `false` value.

There's more...

The `Thread` class has another method to check whether a thread has been interrupted or not. It's the static method, `interrupted()`, that checks whether the current thread has been interrupted or not.

There is an important difference between the `isInterrupted()` and `interrupted()` methods. The first one doesn't change the value of the interrupted attribute, but the second one sets it to `false`.

As mentioned earlier, a thread object can ignore its interruption, but this is not the expected behavior.

Controlling the interruption of a thread

In the previous recipe, you learned how you can interrupt the execution of a thread and what you have to do to control this interruption in the thread object. The mechanism shown in the previous example can be used if the thread that can be interrupted is simple. But if the thread implements a complex algorithm divided into some methods or it has methods with recursive calls, we will need to use a better mechanism to control the interruption of the thread. Java provides the `InterruptedException` exception for this purpose. You can throw this exception when you detect the interruption of a thread and catch it in the `run()` method.

In this recipe, we will implement a task that will look for files with a determined name in a folder and in all its subfolders. This is to show how you can use the `InterruptedException` exception to control the interruption of a thread.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class called `FileSearch` and specify that it implements the `Runnable` interface:

```
public class FileSearch implements Runnable {
```

2. Declare two private attributes: one for the name of the file we are going to search for and one for the initial folder. Implement the constructor of the class, which initializes these attributes:

```
private String initPath;
private String fileName;
public FileSearch(String initPath, String fileName) {
  this.initPath = initPath;
  this.fileName = fileName;
}
```

3. Implement the `run()` method of the `FileSearch` class. It checks whether the attribute `fileName` is a directory; if it is, it calls the `directoryProcess()` method. This method can throw an `InterruptedException` exception, so we have to catch them:

```
@Override
public void run() {
  File file = new File(initPath);
  if (file.isDirectory()) {
    try {
      directoryProcess(file);
    } catch (InterruptedException e) {
      System.out.printf("%s: The search has been interrupted",
                        Thread.currentThread().getName());
    }
  }
}
```

4. Implement the `directoryProcess()` method. This method will obtain the files and subfolders in a folder and process them. For each directory, the method will make a recursive call, passing the directory as a parameter. For each file, the method will call the `fileProcess()` method. After processing all files and folders, the method checks whether the thread has been interrupted; if yes, as in this case, it will throw an `InterruptedException` exception:

```
private void directoryProcess(File file) throws
                          InterruptedException {
  File list[] = file.listFiles();
  if (list != null) {
    for (int i = 0; i < list.length; i++) {
      if (list[i].isDirectory()) {
        directoryProcess(list[i]);
      } else {
        fileProcess(list[i]);
      }
    }
  }
  if (Thread.interrupted()) {
    throw new InterruptedException();
  }
}
```

5. Implement the `fileProcess()` method. This method will compare the name of the file it's processing with the name we are searching for. If the names are equal, we will write a message in the console. After this comparison, the thread will check whether it has been interrupted; if yes, as in this case, it will throw an `InterruptedException` exception:

```
private void fileProcess(File file) throws
                            InterruptedException {
  if (file.getName().equals(fileName)) {
    System.out.printf("%s : %s\n",
                      Thread.currentThread().getName(),
                      file.getAbsolutePath());
  }
  if (Thread.interrupted()) {
    throw new InterruptedException();
  }
}
```

6. Now let's implement the main class of the example. Implement a class called `Main` that contains the `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

7. Create and initialize an object of the `FileSearch` class and thread to execute its task. Then start executing the thread. I have used a Windows operating system route. If you work with other operating systems, such as Linux or iOS, change the route to the one that exists on your operating system:

```
FileSearch searcher = new FileSearch("C:\\Windows",
                                     "explorer.exe");
Thread thread=new Thread(searcher);
thread.start();
```

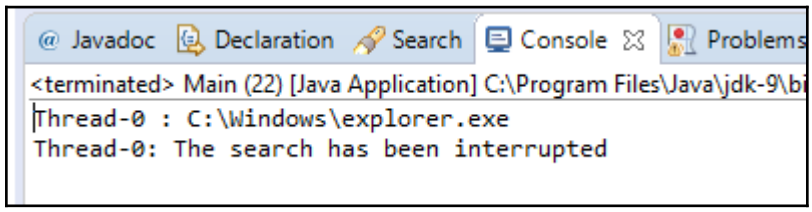
8. Wait for 10 seconds and interrupt the thread:

```
  try {
    TimeUnit.SECONDS.sleep(10);
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
  thread.interrupt();
}
```

9. Run the example and see the results.

How it works...

The following screenshot shows the result of an execution of this example. You can see how the `FileSearch` object ends its execution when it detects that it has been interrupted.



In this example, we use Java exceptions to control the interruption of a thread. When you run the example, the program starts going through folders by checking whether they have the file or not. For example, if you enter in the `\b\c\d` folder, the program will have three recursive calls to the `directoryProcess()` method. When it detects that it has been interrupted, it throws an `InterruptedException` exception and continues the execution in the `run()` method, no matter how many recursive calls have been made.

There's more...

The `InterruptedException` exception is thrown by some Java methods related to a concurrency API, such as `sleep()`. In this case, this exception is thrown if the thread is interrupted (with the `interrupt()` method) when it's sleeping.

See also

- The *Interrupting a thread* recipe of this chapter

Sleeping and resuming a thread

Sometimes, you may be interested in pausing the execution of a thread during a determined period of time. For example, a thread in a program checks the sensor state once per minute. The rest of the time, it does nothing. During this time, the thread doesn't use any resources of the computer. After this period is over, the thread will be ready to continue with its execution when the operating system scheduler chooses it to be executed. You can use the `sleep()` method of the `Thread` class for this purpose. This method receives a long number as a parameter that indicates the number of milliseconds during which the thread will suspend its execution. After that time, the thread continues with its execution in the next instruction to the `sleep()` one when the JVM assigns it CPU time.

Another possibility is to use the `sleep()` method of an element of the `TimeUnit` enumeration. This method uses the `sleep()` method of the `Thread` class to put the current thread to sleep, but it receives the parameter in the unit that it represents and converts it into milliseconds.

In this recipe, we will develop a program that uses the `sleep()` method to write the actual date every second.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class called `ConsoleClock` and specify that it implements the `Runnable` interface:

```
public class ConsoleClock implements Runnable {
```

2. Implement the `run()` method:

```
@Override
public void run() {
```

3. Write a loop with 10 iterations. In each iteration, create a `Date` object, write it to the console, and call the `sleep()` method of the `SECONDS` attribute of the `TimeUnit` class to suspend the execution of the thread for 1 second. With this value, the thread will be sleeping for approximately 1 second. As the `sleep()` method can throw an `InterruptedException` exception, we have to include some code to catch it. It's good practice to include code that frees or closes the resources the thread is using when it's interrupted:

```
    for (int i = 0; i < 10; i++) {
      System.out.printf("%s\n", new Date());
      try {
        TimeUnit.SECONDS.sleep(1);
      } catch (InterruptedException e) {
        System.out.printf("The FileClock has been interrupted");
      }
    }
  }
```

4. We have implemented the thread. Now let's implement the main class of the example. Create a class called `Main` that contains the `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

5. Create an object of the `FileClock` class and a `thread` to execute it. Then, start executing a thread:

```
FileClock clock=new FileClock();
Thread thread=new Thread(clock);
thread.start();
```

6. Call the `sleep()` method of the `SECONDS` attribute of the `TimeUnit` class in the main thread to wait for 5 seconds:

```
try {
  TimeUnit.SECONDS.sleep(5);
} catch (InterruptedException e) {
  e.printStackTrace();
};
```

7. Interrupt the `FileClock` thread:

```
thread.interrupt();
```

8. Run the example and see the results.

How it works...

When you run the example, you would see how the program writes a `Date` object per second and also the message indicating that the `FileClock` thread has been interrupted.

When you call the `sleep()` method, the thread leaves the CPU and stops its execution for a period of time. During this time, it's not consuming CPU time, so the CPU could be executing other tasks.

When the thread is sleeping and is interrupted, the method throws an `InterruptedException` exception immediately and doesn't wait until the sleeping time is finished.

There's more...

The Java concurrency API has another method that makes a thread object leave the CPU. It's the `yield()` method, which indicates to the JVM that the thread object can leave the CPU for other tasks. The JVM does not guarantee that it will comply with this request. Normally, it's only used for debugging purposes.

Waiting for the finalization of a thread

In some situations, we will have to wait for the end of the execution of a thread (the `run()` method ends its execution). For example, we may have a program that will begin initializing the resources it needs before proceeding with the rest of the execution. We can run initialization tasks as threads and wait for their finalization before continuing with the rest of the program.

For this purpose, we can use the `join()` method of the `Thread` class. When we call this method using a thread object, it suspends the execution of the calling thread until the object that is called finishes its execution.

In this recipe, we will learn the use of this method with an initialization example.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class called `DataSourcesLoader` and specify that it implements the `Runnable` interface:

```
public class DataSourcesLoader implements Runnable {
```

2. Implement the `run()` method. It writes a message to indicate that it starts its execution, sleeps for 4 seconds, and writes another message to indicate that it ends its execution:

```
@Override
public void run() {
  System.out.printf("Beginning data sources loading: %s\n",
                    new Date());
  try {
    TimeUnit.SECONDS.sleep(4);
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
```

```
    System.out.printf("Data sources loading has finished: %s\n",
                      new Date());
  }
```

3. Create a class called `NetworkConnectionsLoader` and specify that it implements the `Runnable` interface. Implement the `run()` method. It will be equal to the `run()` method of the `DataSourcesLoader` class, but it will sleep for 6 seconds.
4. Now, create a class called `Main` that contains the `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

5. Create an object of the `DataSourcesLoader` class and a thread to run it:

```
DataSourcesLoader dsLoader = new DataSourcesLoader();
Thread thread1 = new Thread(dsLoader,"DataSourceThread");
```

6. Create an object of the `NetworkConnectionsLoader` class and a thread to run it:

```
NetworkConnectionsLoader ncLoader = new NetworkConnectionsLoader();
Thread thread2 = new Thread(ncLoader,"NetworkConnectionLoader");
```

7. Call the `start()` method of both the thread objects:

```
thread1.start();
thread2.start();
```

8. Wait for the finalization of both the threads using the `join()` method. This method can throw an `InterruptedException` exception, so we have to include the code to catch it:

```
try {
   thread1.join();
   thread2.join();
 } catch (InterruptedException e) {
   e.printStackTrace();
 }
```

9. Write a message to indicate the end of the program:

```
System.out.printf("Main: Configuration has been loaded: %s\n",
                  new Date());
```

10. Run the program and see the results.

How it works...

When you run this program, you would understand how both the thread objects start their execution. First, the `DataSourcesLoader` thread finishes its execution. Then, the `NetworkConnectionsLoader` class finishes its execution. At this moment, the `main` thread object continues its execution and writes the final message.

There's more...

Java provides two additional forms of the `join()` method:

- `join (long milliseconds)`
- `join (long milliseconds, long nanos)`

In the first version of the `join()` method, instead of indefinitely waiting for the finalization of the thread called, the calling thread waits for the milliseconds specified as the parameter of the method. For example, if the object `thread1` has `thread2.join(1000)`, `thread1` suspends its execution until one of these two conditions are met:

- `thread2` has finished its execution
- 1,000 milliseconds have passed

When one of these two conditions is `true`, the `join()` method returns. You can check the status of the thread to know whether the `join()` method was returned because it finished its execution or because the specified time had passed.

The second version of the `join()` method is similar to the first one, but it receives the number of milliseconds and nanoseconds as parameters.

Creating and running a daemon thread

Java has a special kind of thread called **daemon** thread. When daemon threads are the only threads running in a program, the JVM ends the program after finishing these threads.

With these characteristics, daemon threads are normally used as service providers for normal (also called **user**) threads running in the same program. They usually have an infinite loop that waits for the service request or performs the tasks of a thread. A typical example of these kinds of threads is the Java garbage collector.

In this recipe, we will learn how to create a daemon thread by developing an example with two threads: one user thread that would write events on a queue and a daemon thread that would clean the queue, removing the events that were generated more than 10 seconds ago.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create the `Event` class. This class only stores information about the events our program will work with. Declare two private attributes: one called the date of the `java.util.Date` type and the other called the event of the `String` type. Generate the methods to write and read their values.
2. Create the `WriterTask` class and specify that it implements the `Runnable` interface:

```
public class WriterTask implements Runnable {
```

3. Declare the queue that stores the events and implement the constructor of the class that initializes this queue:

```
private Deque<Event> deque;
public WriterTask (Deque<Event> deque){
  this.deque=deque;
}
```

4. Implement the `run()` method of this task. This method will have a loop with 100 iterations. In each iteration, we create a new event, save it in the queue, and sleep for 1 second:

```
@Override
public void run() {
  for (int i=1; i<100; i++) {
    Event event=new Event();
    event.setDate(new Date());
    event.setEvent(String.format("The thread %s has generated
                   an event", Thread.currentThread().getId()));
    deque.addFirst(event);
```

```
      try {
        TimeUnit.SECONDS.sleep(1);
      } catch (InterruptedException e) {
        e.printStackTrace();
      }
    }
  }
```

5. Create the `CleanerTask` class and specify that it extends the `Thread` class:

```
public class CleanerTask extends Thread {
```

6. Declare the queue that stores the events and implement the constructor of the class that initializes this queue. In the constructor, mark this thread as a daemon thread with the `setDaemon()` method:

```
private Deque<Event> deque;
public CleanerTask(Deque<Event> deque) {
  this.deque = deque;
  setDaemon(true);
}
```

7. Implement the `run()` method. It has an infinite loop that gets the actual date and calls the `clean()` method:

```
@Override
public void run() {
  while (true) {
    Date date = new Date();
    clean(date);
  }
}
```

8. Implement the `clean()` method. It gets the last event, and if it was created more than 10 seconds ago, it deletes it and checks the next event. If an event is deleted, it writes the message of the event and the new size of the queue so you can see its evolution:

```
private void clean(Date date) {
  long difference;
  boolean delete;

  if (deque.size()==0) {
   return;
  }
  delete=false;
  do {
```

```
        Event e = deque.getLast();
        difference = date.getTime() - e.getDate().getTime();
        if (difference > 10000) {
          System.out.printf("Cleaner: %s\n",e.getEvent());
          deque.removeLast();
          delete=true;
        }
      } while (difference > 10000);
      if (delete){
        System.out.printf("Cleaner: Size of the queue: %d\n",
                          deque.size());
      }
    }
```

9. Now implement the `main` class. Create a class called `Main` with a `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

10. Create the queue to store the events using the `Deque` class:

```
Deque<Event> deque=new ConcurrentLinkedDeque<Event>();
```

11. Create and start as many `WriterTask` threads as available processors have the JVM and one `CleanerTask` method:

```
WriterTask writer=new WriterTask(deque);
for (int i=0; i< Runtime.getRuntime().availableProcessors();
     i++){
  Thread thread=new Thread(writer);
  thread.start();
}
CleanerTask cleaner=new CleanerTask(deque);
cleaner.start();
```

12. Run the program and see the results.

How it works...

If you analyze the output of one execution of the program, you would see how the queue begins to grow until it has a size of, in our case, `40` events. Then, its size will vary around `40` events it has grown up to until the end of the execution. This size may depend on the number of cores of your machine. I have executed the code in a four-core processor, so we launch four `WriterTask` tasks.

The program starts with four `WriterTask` threads. Each thread writes an event and sleeps for 1 second. After the first `10` seconds, we have `40` events in the queue. During these `10` seconds, `CleanerTask` are executed whereas the four `WriterTask` threads sleep; however, but it doesn't delete any event because all of them were generated less than `10` seconds ago. During the rest of the execution, `CleanerTask` deletes four events every second and the four `WriterTask` threads write another four; therefore, the size of the queue varies around `40` events it has grown up to. Remember that the execution of this example depends on the number of available cores to the JVM of your computer. Normally, this number is equal to the number of cores of your CPU.

You can play with time until the `WriterTask` threads are sleeping. If you use a smaller value, you will see that `CleanerTask` has less CPU time and the size of the queue will increase because `CleanerTask` doesn't delete any event.

There's more...

You only can call the `setDaemon()` method before you call the `start()` method. Once the thread is running, you can't modify its daemon status calling the `setDaemon()` method. If you call it, you will get an `IllegalThreadStateException` exception.

You can use the `isDaemon()` method to check whether a thread is a daemon thread (the method returns `true`) or a non-daemon thread (the method returns `false`).

Processing uncontrolled exceptions in a thread

A very important aspect in every programming language is the mechanism that helps you manage error situations in your application. The Java programming language, as almost all modern programming languages, implements an exception-based mechanism to manage error situations. These exceptions are thrown by Java classes when an error situation is detected. You can also use these exceptions or implement your own exceptions to manage the errors produced in your classes.

Java also provides a mechanism to capture and process these exceptions. There are exceptions that must be captured or re-thrown using the `throws` clause of a method. These exceptions are called checked exceptions. There are exceptions that don't have to be specified or caught. These are unchecked exceptions:

- **Checked exceptions**: These must be specified in the `throws` clause of a method or caught inside them, for example, `IOException` or `ClassNotFoundException`.
- **Unchecked exceptions**: These don't need to be specified or caught, for example, `NumberFormatException`.

When a checked exception is thrown inside the `run()` method of a thread object, we have to catch and treat them because the `run()` method doesn't accept a `throws` clause. When an unchecked exception is thrown inside the `run()` method of a thread object, the default behavior is to write the stack trace in the console and exit the program.

Fortunately, Java provides us with a mechanism to catch and treat unchecked exceptions thrown in a thread object to avoid ending the program.

In this recipe, we will learn this mechanism using an example.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. First of all, we have to implement a class to treat unchecked exceptions. This class must implement the `UncaughtExceptionHandler` interface and implement the `uncaughtException()` method declared in this interface. It's an interface enclosed in the `Thread` class. In our case, let's call this class `ExceptionHandler` and create a method to write information about `Exception` and `Thread` that threw it. The following is the code:

```
public class ExceptionHandler implements UncaughtExceptionHandler {
  @Override
  public void uncaughtException(Thread t, Throwable e) {
```

```
        System.out.printf("An exception has been captured\n");
        System.out.printf("Thread: %s\n",t.getId());
        System.out.printf("Exception: %s: %s\n",
                          e.getClass().getName(),e.getMessage());
        System.out.printf("Stack Trace: \n");
        e.printStackTrace(System.out);
        System.out.printf("Thread status: %s\n",t.getState());
      }
    }
```

2. Now implement a class that throws an unchecked exception. Call this class `Task`, specify that it implements the `Runnable` interface, implement the `run()` method, and force the exception; for example, try to convert a `String` value into an `int` value:

```
public class Task implements Runnable {
  @Override
  public void run() {
    int numero=Integer.parseInt("TTT");
  }
}
```

3. Now implement the main class of the example. Implement a class called `Main` with its `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

4. Create a `Task` object and `Thread` to run it. Set the unchecked exception handler using the `setUncaughtExceptionHandler()` method and start executing the thread:

```
      Task task=new Task();
      Thread thread=new Thread(task);
      thread.setUncaughtExceptionHandler(new ExceptionHandler());
      thread.start();
    }
  }
```

5. Run the example and see the results.

How it works...

In the following screenshot, you can see the results of the execution of the example. The exception is thrown and captured by the handler that writes the information about `Exception` and `Thread` that threw it. This information is presented in the console:

```
<terminated> Main (3) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe (26 mar. 2017 3:07:07)
An exception has been captured
Thread: 13
Exception: java.lang.NumberFormatException: For input string: "TTT"
Stack Trace:
java.lang.NumberFormatException: For input string: "TTT"
        at java.lang.NumberFormatException.forInputString(java.base@9-ea/NumberFormatException.java:65)
        at java.lang.Integer.parseInt(java.base@9-ea/Integer.java:695)
        at java.lang.Integer.parseInt(java.base@9-ea/Integer.java:813)
        at com.packtpub.java9.concurrency.cookbook.chapter01.recipe07.task.Task.run(Task.java:15)
        at java.lang.Thread.run(java.base@9-ea/Thread.java:843)
Thread status: RUNNABLE
Thread has finished
```

When an exception is thrown in a thread and remains uncaught (it has to be an unchecked exception), the JVM checks whether the thread has an uncaught exception handler set by the corresponding method. If it does, the JVM invokes this method with the `Thread` object and `Exception` as arguments.

If the thread doesn't have an uncaught exception handler, the JVM prints the stack trace in the console and ends the execution of the thread that had thrown the exception.

There's more...

The `Thread` class has another method related to the process of uncaught exceptions. It's the static method `setDefaultUncaughtExceptionHandler()` that establishes an exception handler for all the thread objects in the application.

When an uncaught exception is thrown in the thread, the JVM looks for three possible handlers for this exception.

First it looks for the uncaught exception handler of the thread objects, as we learned in this recipe. If this handler doesn't exist, the JVM looks for the uncaught exception handler of `ThreadGroup` as explained in the *Grouping threads and processing uncontrolled exceptions in a group of threads* recipe. If this method doesn't exist, the JVM looks for the default uncaught exception handler, as we learned in this recipe.

If none of the handlers exits, the JVM writes the stack trace of the exception in the console and ends the execution of the Thread that had thrown the exception.

See also

- The *Grouping threads and processing uncontrolled exceptions in a group of threads* recipe of this chapter

Using thread local variables

One of the most critical aspects of a concurrent application is shared data. This has special importance in objects that extend the `Thread` class or implement the `Runnable` interface and in objects that are shared between two or more threads.

If you create an object of a class that implements the `Runnable` interface and then start various thread objects using the same `Runnable` object, all the threads would share the same attributes. This means that if you change an attribute in a thread, all the threads will be affected by this change.

Sometimes, you will be interested in having an attribute that won't be shared among all the threads that run the same object. The Java Concurrency API provides a clean mechanism called **thread-local variables** with very good performance. They have some disadvantages as well. They retain their value while the thread is alive. This can be problematic in situations where threads are reused.

In this recipe, we will develop two programs: one that would expose the problem in the first paragraph and another that would solve this problem using the thread-local variables mechanism.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. First, implement a program that has the problem exposed previously. Create a class called `UnsafeTask` and specify that it implements the `Runnable` interface. Declare a private `java.util.Date` attribute:

```
public class UnsafeTask implements Runnable{
  private Date startDate;
```

2. Implement the `run()` method of the `UnsafeTask` object. This method will initialize the `startDate` attribute, write its value to the console, sleep for a random period of time, and again write the value of the `startDate` attribute:

```
@Override
public void run() {
  startDate=new Date();
  System.out.printf("Starting Thread: %s : %s\n",
                    Thread.currentThread().getId(),startDate);
  try {
    TimeUnit.SECONDS.sleep( (int)Math.rint(Math.random()*10));
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
  System.out.printf("Thread Finished: %s : %s\n",
                    Thread.currentThread().getId(),startDate);
}
```

3. Now, implement the main class of this problematic application. Create a class called `Main` with a `main()` method. This method will create an object of the `UnsafeTask` class and start `10` threads using this object, sleeping for 2 seconds between each thread:

```
public class Main {
  public static void main(String[] args) {
    UnsafeTask task=new UnsafeTask();
    for (int i=0; i<10; i++){
      Thread thread=new Thread(task);
      thread.start();
      try {
        TimeUnit.SECONDS.sleep(2);
      } catch (InterruptedException e) {
        e.printStackTrace();
      }
    }
  }
}
```

4. In the following screenshot, you can see the results of this program's execution. Each thread has a different start time, but when they finish, there is a change in the value of the attribute. So they are writing a bad value. For example, check out the thread with the ID 13:

```
<terminated> Main (4) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe
Starting Thread: 13 : Sun Mar 26 03:12:42 CEST 2017
Starting Thread: 14 : Sun Mar 26 03:12:44 CEST 2017
Starting Thread: 15 : Sun Mar 26 03:12:46 CEST 2017
Thread Finished: 14 : Sun Mar 26 03:12:46 CEST 2017
Thread Finished: 13 : Sun Mar 26 03:12:46 CEST 2017
Thread Finished: 15 : Sun Mar 26 03:12:46 CEST 2017
```

5. As mentioned earlier, we are going to use the thread-local variables mechanism to solve this problem.
6. Create a class called `SafeTask` and specify that it implements the `Runnable` interface:

```
public class SafeTask implements Runnable {
```

7. Declare an object of the `ThreadLocal<Date>` class. This object will have an implicit implementation that would include the `initialValue()` method. This method will return the actual date:

```
 private static ThreadLocal<Date> startDate=new
                                        ThreadLocal<Date>(){
  protected Date initialValue(){
    return new Date();
  }
};
```

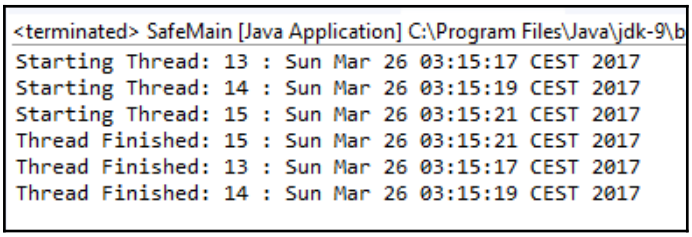
8. Implement the `run()` method. It has the same functionality as the `run()` method of `UnsafeTask class`, but it changes the way it accesses the `startDate` attribute. Now we will use the `get()` method of the `startDate` object:

```
@Override
public void run() {
  System.out.printf("Starting Thread: %s : %s\n",
              Thread.currentThread().getId(),startDate.get());
  try {
    TimeUnit.SECONDS.sleep((int)Math.rint(Math.random()*10));
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
  System.out.printf("Thread Finished: %s : %s\n",
              Thread.currentThread().getId(),startDate.get());
}
```

9. The `Main` class of this example is the same as the unsafe example. The only difference is that it changes the name of the `Runnable` class.
10. Run the example and analyze the difference.

How it works...

In the following screenshot, you can see the results of the execution of the safe sample. The ten `Thread` objects have their own value of the `startDate` attribute:



The thread-local variables mechanism stores a value of an attribute for each thread that uses one of these variables. You can read the value using the `get()` method and change the value using the `set()` method. The first time you access the value of a thread-local variable, if it has no value for the thread object that it is calling, the thread-local variable will call the `initialValue()` method to assign a value for that thread and return the initial value.

There's more...

The thread-local class also provides the `remove()` method that deletes the value stored in a thread-local variable for the thread that it's calling.

The Java Concurrency API includes the `InheritableThreadLocal` class that provides inheritance of values for threads created from a thread. If thread `A` has a value in a thread-local variable and it creates another thread `B`, then thread `B` will have the same value as thread `A` in the thread-local variable. You can override the `childValue()` method that is called to initialize the value of the child thread in the thread-local variable. It receives the value of the parent thread as a parameter in the thread-local variable.

Grouping threads and processing uncontrolled exceptions in a group of threads

An interesting functionality offered by the concurrency API of Java is the ability to group threads. This allows us to treat the threads of a group as a single unit and provide access to the thread objects that belong to a group in order to do an operation with them. For example, you have some threads doing the same task and you want to control them. You can, for example, interrupt all the threads of the group with a single call.

Java provides the `ThreadGroup` class to work with a groups of threads. A `ThreadGroup` object can be formed by thread objects and another `ThreadGroup` object, generating a tree structure of threads.

In the *Controlling the interruption of a thread* recipe, you learned how to use a generic method to process all the uncaught exceptions that are thrown in a thread object. In the *Processing uncontrolled exceptions in a thread* recipe, we wrote a handler to process the uncaught exceptions thrown by a thread. We can use a similar mechanism to process the uncaught exceptions thrown by a thread or a group of threads.

In this recipe, we will learn to work with `ThreadGroup` objects and how to implement and set the handler that would process uncaught exceptions in a group of threads. We'll do this using an example.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. First, extend the `ThreadGroup` class by creating a class called `MyThreadGroup` that would be extended from `ThreadGroup`. You have to declare a constructor with one parameter because the `ThreadGroup` class doesn't have a constructor without it. Extend the `ThreadGroup` class to override the `uncaughtException()` method in order to process the exceptions thrown by the threads of the group:

```
public class MyThreadGroup extends ThreadGroup {
  public MyThreadGroup(String name) {
    super(name);
  }
```

2. Override the `uncaughtException()` method. This method is called when an exception is thrown in one of the threads of the `ThreadGroup` class. In this case, the method will write information about the exception and the thread that throws it; it will present this information in the console. Also, note that this method will interrupt the rest of the threads in the `ThreadGroup` class:

```
@Override
public void uncaughtException(Thread t, Throwable e) {
  System.out.printf("The thread %s has thrown an Exception\n",
                    t.getId());
  e.printStackTrace(System.out);
  System.out.printf("Terminating the rest of the Threads\n");
  interrupt();
}
```

3. Create a class called `Task` and specify that it implements the `Runnable` interface:

```
public class Task implements Runnable {
```

4. Implement the `run()` method. In this case, we will provoke an `AritmethicException` exception. For this, we will divide 1,000 with random numbers until the random generator generates zero to throw the exception:

```
@Override
public void run() {
  int result;
  Random random=new Random(Thread.currentThread().getId());
  while (true) {
    result=1000/((int)(random.nextDouble()*1000000000));
```

```
      if (Thread.currentThread().isInterrupted()) {
        System.out.printf("%d : Interrupted\n",
                          Thread.currentThread().getId());
        return;
      }
    }
  }
```

5. Now, implement the main class of the example by creating a class called `Main` and implement the `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

6. First, calculate the number of threads you are going to launch. We use the `availableProcessors()` method of the `Runtime` class (we obtain the runtime object associated with the current Java application with the static method, called `getRuntime()`, of that class). This method returns the number of processors available to the JVM, which is normally equal to the number of cores of the computer that run the application:

```
int numberOfThreads = 2 * Runtime.getRuntime()
                               .availableProcessors();
```

7. Create an object of the `MyThreadGroup` class:

```
MyThreadGroup threadGroup=new MyThreadGroup("MyThreadGroup");
```

8. Create an object of the `Task` class:

```
Task task=new Task();
```

9. Create the calculated number of `Thread` objects with this `Task` class and start them:

```
for (int i = 0; i < numberOfThreads; i++) {
  Thread t = new Thread(threadGroup, task);
  t.start();
}
```

10. Then, write information about `ThreadGroup` in the console:

```
System.out.printf("Number of Threads: %d\n",
                  threadGroup.activeCount());
System.out.printf("Information about the Thread Group\n");
threadGroup.list();
```

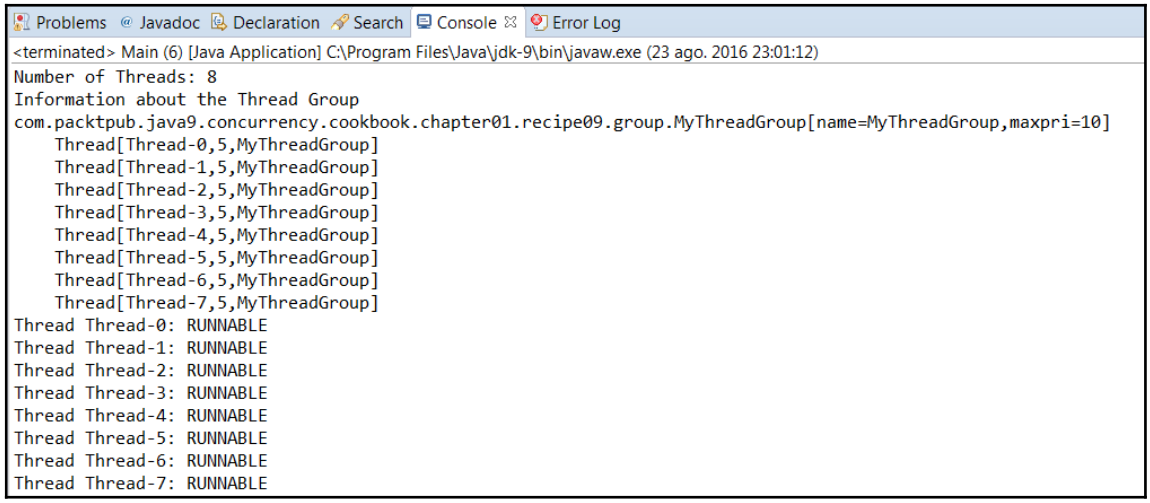
11. Finally, write the status of the threads that form the group:

```
        Thread[] threads = new Thread[threadGroup.activeCount()];
        threadGroup.enumerate(threads);
        for (int i = 0; i < threadGroup.activeCount(); i++) {
          System.out.printf("Thread %s: %s\n", threads[i].getName(),
                            threads[i].getState());
        }
      }
    }
```

12. Run the example and see the results.

How it works...

In the following screenshot, you can see the output of the `list()` method of the `ThreadGroup` class and the output generated when we write the status of each `Thread` object:



The `ThreadGroup` class stores thread objects and other `ThreadGroup` objects associated with it so it can access all of their information (status, for example) and perform operations over all its members (interrupt, for example).

Check out how one of the thread objects threw the exception that interrupted the other objects:

```
The thread 18 has thrown an Exception
java.lang.ArithmeticException: / by zero
        at com.packtpub.java9.concurrency.cookbook.chapter01.recipe09.task.Task.run(Task.java:18)
        at java.lang.Thread.run(java.base@9-ea/Thread.java:843)
Terminating the rest of the Threads
19 : Interrupted
14 : Interrupted
15 : Interrupted
13 : Interrupted
20 : Interrupted
17 : Interrupted
16 : Interrupted
```

When an uncaught exception is thrown in a `Thread` object, the JVM looks for three possible handlers for this exception.

First, it looks for the uncaught exception handler of the thread, as explained in the *Processing uncontrolled exceptions in a thread* recipe. If this handler doesn't exist, then the JVM looks for the uncaught exception handler of the `ThreadGroup` class of the thread, as learned in this recipe. If this method doesn't exist, the JVM looks for the default uncaught exception handler, as explained in the *Processing uncontrolled exceptions in a thread* recipe.

If none of the handlers exists, the JVM writes the stack trace of the exception in the console and ends the execution of the thread that had thrown the exception.

See also

- The *Processing uncontrolled exceptions in a thread* recipe

Creating threads through a factory

The factory pattern is one of the most used design patterns in the object-oriented programming world. It is a creational pattern, and its objective is to develop an object whose mission should be this: creating other objects of one or several classes. With this, if you want to create an object of one of these classes, you could just use the factory instead of using a new operator.

With this factory, we centralize the creation of objects with some advantages:

- It's easy to change the class of the objects created or the way you'd create them.
- It's easy to limit the creation of objects for limited resources; for example, we can only have *n* objects of a given type.
- It's easy to generate statistical data about the creation of objects.

Java provides an interface, the `ThreadFactory` interface, to implement a thread object factory. Some advanced utilities of the Java concurrency API use thread factories to create threads.

In this recipe, you will learn how to implement a `ThreadFactory` interface to create thread objects with a personalized name while saving the statistics of the thread objects created.

Getting ready

The example for this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class called `MyThreadFactory` and specify that it implements the `ThreadFactory` interface:

```
public class MyThreadFactory implements ThreadFactory {
```

2. Declare three attributes: an integer number called counter, which we will use to store the number of thread objects created, a string called name with the base name of every thread created, and a list of string objects called stats to save statistical data about the thread objects created. Also, implement the constructor of the class that initializes these attributes:

```
private int counter;
private String name;
private List<String> stats;

public MyThreadFactory(String name){
  counter=0;
  this.name=name;
```

```
    stats=new ArrayList<String>();
  }
```

3. Implement the `newThread()` method. This method will receive a `Runnable` interface and return a thread object for this `Runnable` interface. In our case, we generate the name of the thread object, create the new thread object, and save the statistics:

```
@Override
public Thread newThread(Runnable r) {
  Thread t=new Thread(r,name+"-Thread_"+counter);
  counter++;
  stats.add(String.format("Created thread %d with name %s on %s\n",
                          t.getId(),t.getName(),new Date()));
  return t;
}
```

4. Implement the `getStatistics()` method; it returns a `String` object with the statistical data of all the thread objects created:

```
public String getStats(){
  StringBuffer buffer=new StringBuffer();
  Iterator<String> it=stats.iterator();

  while (it.hasNext()) {
    buffer.append(it.next());
    buffer.append("\n");
  }

  return buffer.toString();
}
```

5. Create a class called `Task` and specify that it implements the `Runnable` interface. In this example, these tasks are going to do nothing apart from sleeping for 1 second:

```
public class Task implements Runnable {
  @Override
  public void run() {
    try {
      TimeUnit.SECONDS.sleep(1);
    } catch (InterruptedException e) {
      e.printStackTrace();
    }
  }
}
```

6. Create the main class of the example. Create a class called `Main` and implement the `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

7. Create a `MyThreadFactory` object and a `Task` object:

```
MyThreadFactory factory=new MyThreadFactory("MyThreadFactory");
Task task=new Task();
```

8. Create 10 `Thread` objects using the `MyThreadFactory` object and start them:

```
Thread thread;
System.out.printf("Starting the Threads\n");
for (int i=0; i<10; i++){
   thread=factory.newThread(task);
  thread.start();
}
```

9. Write the statistics of the thread factory in the console:

```
System.out.printf("Factory stats:\n");
System.out.printf("%s\n",factory.getStats());
```

10. Run the example and see the results.

How it works...

The `ThreadFactory` interface has only one method, called `newThread()`. It receives a `Runnable` object as a parameter and returns a `Thread` object. When you implement a `ThreadFactory` interface, you have to implement it and override the `newThread` method. The most basic `ThreadFactory` has only one line:

```
return new Thread(r);
```

You can improve this implementation by adding some variants, as follows:

- Creating personalized threads, as in the example, using a special format for the name or even creating your own `Thread` class that would inherit the Java `Thread` class
- Saving thread creation statistics, as shown in the previous example
- Limiting the number of threads created
- Validating the creation of the threads

You can add anything else you can imagine to the preceding list. The use of the factory design pattern is a good programming practice, but if you implement a `ThreadFactory` interface to centralize the creation of threads, you will have to review the code to guarantee that all the threads are created using the same factory.

See also

- The *Implementing the ThreadFactory interface to generate custom threads* and *Using our ThreadFactory in an Executor object* recipes in `Chapter 8`, *Customizing Concurrency Classes*